AMERICAN UNIVERSITY OF BEIRUT

Module 2 - Lecture Notes

From Idea to Backend Implementation with AI

AI-Assisted Coding

Module purpose

In Module 1, you built a FastAPI skeleton with a working /health endpoint. In Module 2, that skeleton becomes a working backend: a strict Pydantic data model, five CRUD endpoints, status-transition business rules, and a pytest suite you prove with a Break Test.

How to use these notes

Use them while watching the Module 2 videos and completing the hands-on guide. Pause after each demo, complete the activity, and do not continue until the verification step passes. The prompt labels, such as A1, B1b, C1, and D1, match the module workflow; the exact prompt text should be kept in the prompt library when that resource is updated.

Learning outcomes

1. Move from browser-based AI support to editor-based AI support inside Cursor IDE.
2. Attach real project files to AI prompts so the assistant works from the actual codebase instead of guessing.
3. Generate and review a strict Pydantic v2 model layer for a Task Tracker API.
4. Create an in-memory storage layer with predictable helper functions.
5. Build five FastAPI CRUD endpoints one at a time and verify each endpoint before moving on.
6. Add backend business rules for task status transitions instead of relying on UI-only guards.
7. Generate, inspect, and run a pytest suite for the API.
8. Perform a Break Test to prove that important tests fail when production code is broken.

Module at a glance: One Repo, Five Connected Parts

The module is one continuous build. Each part creates the foundation for the next prompt and the next verification step.

Part	Focus	Main output	Verification gate
2.0	Define the tool	Cursor can read the repo and explain app/main.py.	/health returns 200.
2.1	Generate the data model	app/models.py and app/storage.py.	tests/verify_a.py prints every check as PASS.
2.2	Scaffold CRUD endpoints	POST, GET list, GET by id, PATCH, DELETE.	curl and /docs confirm expected status codes and schemas.
2.3	Add business logic	app/business_rules.py and PATCH transition validation.	Transition matrix returns 200, 200, 422, 200, 422, 200.
2.4	Write API tests	tests/conftest.py and tests/test_tasks.py.	pytest passes, then the Break Test causes the right tests to fail.

Thread through the whole module

Strict prompt -> review -> verification -> targeted correction. Do not let errors compound. Cursor can draft code, but you own correctness.

Part 1 - Define the Tool: Cursor IDE

Module 2 moves the AI into the editor because the work now happens inside real project files. Browser-based assistants were useful in Module 1 for planning, requirements, architecture, and scaffolding. In this module, the assistant needs access to the actual codebase.

Why the tool matters

- Cursor Chat can work with file references, so prompts can include real files like app/main.py, app/models.py, and app/storage.py.
- File-aware prompting reduces hallucinated project structure because the assistant can inspect the code you attach.

- Apply in Editor lets you insert generated code into the project, but only after you review it.
- Inline completions are useful for boilerplate, but they should not replace deliberate prompting for new logic.

Setup checklist

- Open the Module 1 task-tracker repository as the workspace.
- Open Cursor Chat and confirm it responds.
- Confirm the file reference picker can see your project files. In the recorded demo this is Cursor; the UI may show @ references or file chips depending on setup.
- Attach app/main.py and ask Cursor to explain the existing /health endpoint.
- Run the app and confirm GET /health still returns HTTP 200.

```
uvicorn app.main:app --reload --port 8000
# In another terminal or browser, verify /health returns 200.
```

First verified interaction

Do not continue until Cursor can read app/main.py and correctly explain what the /health endpoint is doing.

Part 2.1 - Generate the Data Model

The data model is the contract every endpoint is built on. If the model is loose or wrong, every endpoint will inherit those mistakes. This is why the first implementation prompt is strict and verification-heavy.

Goal

- Create app/models.py with Pydantic v2 request and response models.
- Create app/storage.py with an in-memory storage layer and predictable helper functions.
- Use enums for task status and priority instead of plain strings.
- Keep server-managed fields, such as id, created_at, and updated_at, out of client input models.

Core model expectations

Element	Expected role
TaskStatus	Enum values such as ToDo, InProgress, and Done.
TaskPriority	Enum values for priority, with Medium as the default used in the demo.
TaskCreate	Client input for creating a task. Requires a non-blank title and rejects extra fields.

Element	Expected role
TaskUpdate	Client input for partial updates. Allows optional editable fields, but not server-managed fields.
TaskResponse	Response model returned by the API. Includes server-generated id and timestamps.
storage helpers	add_task, get_all_tasks, get_task_by_id, update_task, delete_task, and _reset.

Prompt A1: generate model and storage

Prompt A1 should specify exact files, class names, enum values, defaults, validators, storage function names, and Pydantic v2 syntax. Before applying the generated code, inspect it carefully.

- Check that Pydantic v2 syntax is used, such as @field_validator and model_dump where needed.
- Check that status and priority are enums, not unconstrained strings.
- Check that title validation rejects empty and whitespace-only titles.
- Check that extra fields are forbidden in input models.
- Check that id, created_at, and updated_at are generated by storage/API code, not accepted from create/update payloads.

  python -c "from app.models import TaskCreate, TaskUpdate, TaskResponse, TaskStatus, TaskPriority; from app.storage import add_task, get_all_tasks, get_task_by_id, update_task, delete_task, _reset; print('OK')"

Verification A: tests/verify_a.py

The verification script should print PASS for all eight checks below. If any line says FAIL, use Prompt A2 as a targeted fix prompt and rerun the verification before continuing.

1. Whitespace title rejected.
2. Empty title rejected.
3. Title over 200 characters rejected.
4. Defaults applied: status ToDo, priority Medium, and empty description.
5. Extra field rejected on TaskCreate.
6. id rejected on TaskCreate.
7. created_at rejected on TaskUpdate.
8. Invalid status rejected.

```
python -m tests.verify_a
# Expected: every line says PASS.
```

Common AI mistakes to catch

- Mixes Pydantic v1 syntax into a Pydantic v2 project.
- Uses plain strings for status instead of TaskStatus.
- Accepts whitespace-only titles because it does not strip before checking.
- Allows clients to send id or timestamps in TaskCreate or TaskUpdate.
- Omits extra="forbid", allowing arbitrary fields silently.
- Invents helper names that later endpoint prompts do not use.

What you leave Part 2.1 with

- app/models.py with strict Pydantic v2 models and enums.
- app/storage.py with in-memory storage helpers and _reset.
- tests/verify_a.py saved and passing all eight checks.
- At least one note in your correction log if Cursor required a fix.

Part 2.2 - Scaffold CRUD Endpoints

This is where the API comes alive. The key teaching rule is simple: one endpoint per prompt, one verification before moving on. Asking for all endpoints at once makes the output harder to review and allows mistakes to multiply.

Prompt quality experiment: vague vs. strict

The demo first submits a vague prompt for POST /tasks and does not apply it. The point is to review what vague prompting misses. Then the strict prompt is submitted with the relevant files attached.

Prompt type	Example	What to notice
Vague	Write a POST /tasks endpoint using FastAPI.	May return 200 instead of 201, invent models, miss imports, or create a new FastAPI app.
Strict	Attach models/storage, specify POST /tasks -> 201, response model, imports, and DO-NOT list.	Produces code that can be inspected and verified against exact expectations.

Lesson from the comparison

Vague prompts can create plausible code. Strict prompts create code you can verify.

Endpoint build order

Order	Endpoint	Expected behavior	Verification focus
1	POST /tasks	Create a task and return 201.	Valid body returns 201; invalid body returns 422.
2	GET /tasks	Return a list of tasks. Optional status and priority filters.	Empty or no-match list returns 200 with [].
3	GET /tasks/{task_id}	Return one task if found.	Found returns 200; missing id returns 404.
4	PATCH /tasks/{task_id}	Partially update an existing task.	Found returns 200; missing id returns 404; invalid payload returns 422.
5	DELETE /tasks/{task_id}	Delete an existing task.	Existing task returns 204 with no body; missing id returns 404.

```
uvicorn app.main:app --reload --port 8000
# Verify each route with curl, then open http://localhost:8000/docs to inspect Swagger UI.
```

Review checklist before applying each route

- Uses the existing FastAPI app; it does not create a new FastAPI() instance.
- Uses the existing models and storage helpers; it does not invent new helper names.
- Uses the exact status code expected for that route.
- Uses response_model where appropriate.
- Does not add transition business logic to PATCH yet; that belongs in Part 2.3.
- Preserves Pydantic v2 patterns.

Common AI mistakes to catch

- POST returns 200 instead of 201.
- DELETE returns JSON even though 204 should have an empty body.
- GET /tasks returns 404 for an empty filter result instead of 200 with an empty list.
- The assistant hallucinates storage.filter or storage.find_one even though storage.py does not define those.
- The assistant calls .dict() instead of .model_dump() in Pydantic v2 code.
- The assistant wraps too much code in try/except and hides FastAPI/Pydantic validation errors.

What you leave Part 2.2 with

- Five working CRUD endpoints, added one at a time.
- curl or Swagger verification for every endpoint.
- An end-to-end sequence: create, read, update, list filtered, delete, confirm missing.
- A prompt comparison log showing what the vague POST prompt missed and what the strict prompt fixed.

Part 2.3 - Add Business Logic: Status Transitions

Cursor can easily write a PATCH route that sets a field. That is not the same as implementing a business rule. In this part, the backend must decide whether a requested status change is allowed based on the current status and the requested new status.

Why this belongs in the backend

UI buttons can hide invalid actions, but a user can still call the API directly with curl, Postman, or another client. The backend is the trusted enforcement point.

Transition rules

Transition	Valid?	Reason
ToDo -> InProgress	Yes	Work has started.
InProgress -> Done	Yes	Work is finished.
Done -> InProgress	Yes	A completed task can be reopened.
ToDo -> Done	No	Cannot skip InProgress.
Done -> ToDo	No	Cannot revert to the beginning.
Same status -> same status	No	No-op moves are rejected in the intended rule set.

Prompt C1: create the rule module and update PATCH

- Create app/business_rules.py.
- Define VALID_TRANSITIONS as a frozenset of allowed pairs.
- Define validate_status_transition(current, new).
- Update the existing PATCH route so it calls the validator only when payload.status is provided.
- Return 422 for invalid transitions.
    - Review that the validator checks the pair (current, new), not just whether new is a valid enum value.

- Review that same-to-same transitions are not included in VALID_TRANSITIONS. If Cursor includes them, remove them.
- Review that the route fetches the existing task first so a missing id returns 404 before transition validation.
- Review that title-only or description-only updates skip transition validation.

Verification C: six-test matrix

Run a transition sequence against a created task. The intended status-code pattern is:

200, 200, 422, 200, 422, 200

- T1: ToDo -> InProgress returns 200.
- T2: InProgress -> Done returns 200.
- T3: Done -> ToDo returns 422.
- T4: Done -> InProgress returns 200.
- T5: InProgress -> InProgress returns 422.
- T6: title-only update, no status field, returns 200.

Common AI mistakes to catch

- Allows every enum value because it checks only whether the new status is valid.
- Returns 400 instead of the expected 422.
- Validates status before confirming the task exists, causing the wrong error for missing ids.
- Runs transition validation even when status is not in the PATCH payload.
- Allows same-to-same transitions because the generated set includes them or because the rule was not explicit enough.

What you leave Part 2.3 with

- app/business_rules.py with VALID_TRANSITIONS and validate_status_transition.
- PATCH updated to enforce transitions only when status is being changed.
- The six-test matrix returns the expected pattern.
- Notes about any AI-generated assumptions you corrected.

Part 2.4 - Write API Tests

The testing section is the climax of the module. Cursor can generate tests with good names and plausible assertions, but a passing test suite is not enough. A test is only useful if it fails when the behavior it protects is broken.

Core testing principle

A test that passes when the code is broken is lying.

Prompt D1: generate pytest files

- Generate tests/conftest.py with a TestClient fixture.
- Use an autouse reset fixture so every test starts with empty storage.
- Generate tests/test_tasks.py with 16 or more named tests covering CRUD, validation, transition rules, and delete behavior.
- Use fixtures such as client and created_task to keep tests readable.

Review before running pytest

- Missing title test sends {}, not a blank title string.
- Blank title test sends spaces, such as {"title": " "}.
- Create task test expects 201.
- Delete existing task test expects 204 and asserts r.content == b"" instead of calling r.json().
- Invalid transition test checks an actually invalid transition, such as ToDo -> Done.
- The storage reset fixture has autouse=True so tests do not leak state.

```
pip install "pytest>=8" "httpx>=0.27"
pytest tests/ -v
# Expected: 16 or more tests, all PASSED.
```

Expected test coverage

Area	Examples
Create	Valid task returns 201; missing title and blank title return 422.
List	Empty list returns 200; filters by status or priority return 200.
Get by id	Existing task returns 200; missing id returns 404.
Patch	Valid updates return 200; missing id returns 404; invalid payloads return 422.
Status transitions	Valid transition returns 200; invalid ToDo -> Done returns 422.
Delete	Existing task returns 204 with no body; missing id returns 404.

The Break Test

After the suite is green, deliberately break production code and rerun pytest. This proves that the tests are actually protecting important behavior.

1. Comment out the validate_status_transition call in the PATCH route.
2. Run pytest. The invalid-transition test should fail, typically because it expected 422 but got 200.
3. Restore the validator call and rerun pytest. The suite should pass again.
4. Then break the title validator temporarily, for example by making the blank-title check ineffective.
5. Run pytest. The blank-title test should fail. Restore the validator and rerun the full suite.

> **What to do if the Break Test does not fail**
> Fix the test, not the production code. A test that passes while the relevant behavior is broken is not protecting anything.

Common AI mistakes in generated tests

- Asserts 200 when the endpoint correctly returns 201 or 204.
- Test name says missing_title but the body still contains a title key.
- Calls r.json() on a 204 response.
- Uses async test patterns when the suite is synchronous.
- Forgets autouse=True on the reset fixture, causing tests to share state.
- Uses the wrong id type or a hard-coded id that does not exist.

What you leave Part 2.4 with

- tests/conftest.py with storage reset, TestClient, and helper fixtures.
- tests/test_tasks.py with 16 or more named tests.
- A green pytest run.
- Break Test proof that transition and title validation tests fail when the corresponding production code is broken.
- A short note describing what the tests caught and what, if anything, you had to correct.

Module 2 Deliverables

Before moving on, make sure these deliverables are saved in your repo and notes.

Code deliverables

- app/models.py: Pydantic v2 models, TaskStatus and TaskPriority enums, validators, and extra-field rejection.

- app/storage.py: in-memory storage with add_task, get_all_tasks, get_task_by_id, update_task, delete_task, and _reset.
- app/main.py: five CRUD routes wired to storage, with PATCH calling the transition validator.
- app/business_rules.py: VALID_TRANSITIONS and validate_status_transition.
- tests/verify_a.py: model verification script with all eight checks passing.
- tests/conftest.py and tests/test_tasks.py: 16 or more pytest tests.

Verification deliverables

- GET /health returns 200 before you begin editing.
- python -m tests.verify_a prints PASS for all eight model checks.
- curl or Swagger checks verify all five CRUD endpoints.
- The status-transition matrix returns 200, 200, 422, 200, 422, 200.
- pytest tests/ -v returns a full green suite.
- Break Test evidence: at least the invalid-transition test fails when transition validation is removed, and the blank-title test fails when title validation is broken.

Documentation deliverables

- Prompt comparison log: vague POST prompt vs strict POST prompt, with notes on what changed.
- Correction log: examples of AI output you inspected and fixed.
- Reflection log: 3-5 sentences on where Cursor helped, where it guessed, and what your review caught.

Closing and Bridge to Module 3

Stop for a moment and look at what Module 2 adds. In Module 1, you had a skeleton with a single health endpoint. Now you have a working backend with a strict data model, five CRUD endpoints, transition business rules, and a test suite you proved by breaking the code and watching the right tests fail.

Cursor wrote a lot of the code. But the prompts were yours, the file references were yours, the verification steps were yours, and the final judgment was yours. The AI drafted the backend; you made it correct.

In Module 3, you will build the frontend, connect it to this API, and debug across the frontend-backend boundary. The same loop applies: ask, inspect, run, test, refine.